

Hi3519DV500 SDK 编译指南

1. 环境信息

项目	说明
SDK 版本	Hi3519DV500_SDK_V1.0.2.0
芯片	Hi3519DV500 (ARM Cortex-A55, arm64)
启动介质	eMMC
C 库	musl
交叉编译器	aarch64-v01c01-linux-musl-gcc (V12CS61.011.010)

1.1 目录约定

本文档使用以下变量表示 SDK 相关路径，请根据实际部署位置替换：

```
# SDK 解压根目录（用户自行替换）
SDK_ROOT=<你的SDK解压路径>

# 例如：
# SDK_ROOT=/home/user/Hi3519DV500_SDK_V1.0.2.0
```

后续文档中所有路径均基于 SDK_ROOT 描述：

用途	路径
SDK 根目录	<code>\${SDK_ROOT}/</code>
BSP 编译入口	<code>\${SDK_ROOT}/package/smp/a55_linux/source/bsp/</code>
开源组件源码	<code>\${SDK_ROOT}/package/open_source/</code>
编译产物输出	<code>\${SDK_ROOT}/package/smp/a55_linux/source/bsp/pub/</code>
交叉编译工具链	<code>/opt/linux/x86-arm/aarch64-v01c01-linux-musl-gcc/</code>

1.2 SDK 解包

首次使用需解包 SDK：

```
cd ${SDK_ROOT}
chmod +x sdk.unpack
./sdk.unpack
```

该脚本会解压 `package/*.tgz` 中的所有组件包到对应目录。

2. 前置准备

以下操作均在 Docker 容器（或编译服务器）内执行。

2.1 安装宿主机依赖包

以 root 用户执行：

```
apt-get update && apt-get install -y \
    autoconf automake libtool \
    autopoint gettext \
    po4a \
    gperf \
    python3-pip
```

包名	用途
autoconf automake libtool	gzip 组件构建需要 autoreconf
autopoint gettext	util-linux 组件构建需要
po4a	xz 组件构建翻译文档需要
gperf	eudev 组件编译需要
python3-pip	安装 Python 依赖

2.2 安装 Python 依赖

```
pip install pycryptodome
```

SDK 的 `image_tool` 在打包 `boot_image.bin` 时依赖 `Crypto` 模块。

2.3 补充缺失的内核源码

SDK 分发包缺少 Linux 内核源码压缩包，需手动下载：

```
cd ${SDK_ROOT}/package/open_source/linux/
wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.10.221.tar.gz
```

文件约 175MB，下载可能需要几分钟。

2.4 修改 eudev 编译配置

eudev 默认依赖 `libblkid`，但 SDK 的交叉编译工具链中未提供该库。需在 eudev 的 Makefile 中添加 `--disable-blkid` 参数。

文件路径：`${SDK_ROOT}/package/open_source/eudev/Makefile`

找到 `hi3519dv500` 分支的 `configure` 命令（约第 32 行），将：

```
LDFLAGS="${OSDRV_CROSS_LDFLAGS}" --disable-introspection --disable-selinux 1>/dev/null;
```

改为:

```
LDFLAGS="${OSDRV_CROSS_LDFLAGS}" --disable-introspection --disable-selinux --disable-blkid  
1>/dev/null;
```

3. 编译命令

所有编译命令须在 BSP 目录下执行:

```
cd ${SDK_ROOT}/package/smp/a55_linux/source/bsp
```

3.1 完整编译

```
make all
```

3.2 部分编译

SDK Makefile 支持单独编译各模块:

目标	说明	命令
boot	u-boot + regbin	make boot
atf	ARM Trusted Firmware (bl31)	make atf
kernel	Linux 内核	make kernel
busybox	BusyBox 根文件系统工具	make busybox
pctools	PC 端工具 (mkfs.jffs2 等)	make pctools
boardtools	板端工具 (mtd-utils, eudev 等)	make boardtools
rootfs_build	根文件系统镜像打包	make rootfs_build

3.3 清理

```
make clean      # 清理所有编译产物  
make distclean  # 清理编译产物 + pub 输出目录
```

3.4 可选参数

参数	默认值	说明
BOOT_MEDIA	emmc	启动介质: emmc / spi

参数	默认值	说明
<code>LIB_TYPE</code>	<code>musl</code>	C 库类型: <code>musl</code> / <code>glibc</code>
<code>LLVM</code>	空	使用 LLVM 编译 (设为任意非空值启用)
<code>TEE_ENABLE</code>	<code>0</code>	是否启用 OP-TEE (<code>0</code> / <code>1</code>)

示例:

```
make all BOOT_MEDIA=emmc LIB_TYPE=musl TEE_ENABLE=0
```

4. 编译流程 (make all 详解)

`make all` 按以下顺序依次执行 12 个 task:

Task	名称	说明
0.0	toolchain_version_check	检查交叉编译器版本
0.1	prepare	创建输出目录
0.2	regbin_prepare	生成寄存器配置 bin (从 .xlsb)
1	boot	编译 u-boot-2022.07
2	atf	编译 arm-trusted-firmware-2.7 (bl31.bin)
3	kernel	编译 linux-5.10.221 内核 (ulmage-fdt)
4	secure_libs	编译安全库
5	rootfs_prepare	准备根文件系统目录
6	busybox	编译 busybox-1.34.1
7	pctools	编译 PC 端工具 (mkfs.cramfs, mkfs.jffs2, mkfs.ubifs 等)
8	boardtools	编译板端工具 (mtd-utils, eudev, ethtool, e2fsprogs)
9	rootfs_build	打包根文件系统镜像 (jffs2/ubifs/ext4)
10	teeos / tee_client	OP-TEE (TEE_ENABLE=0 时跳过)
11	gslboot_build	GSL 启动代码 + 打包 boot_image.bin
12	uboot_env	生成 u-boot 环境变量 bin

5. 编译产物

产物均位于 `${SDK_ROOT}/package/smp/a55_linux/source/bsp/pub/` 目录下。

5.1 启动镜像

路径: `pub/hi3519dv500_emmc_image_musl/`

文件	大小	说明
<code>boot_image.bin</code>	~361KB	完整启动镜像 (含 regbin + u-boot + gsl)
<code>bl31.bin</code>	~29KB	ARM Trusted Firmware
<code>uImage-fdt</code>	~12MB	Linux 内核 (含设备树)
<code>emmc_env.bin</code>	~256KB	u-boot 环境变量
<code>emmc_burn_table.xml</code>	~1KB	eMMC 烧写分区表

5.2 根文件系统镜像

路径: `pub/hi3519dv500_emmc_image_musl/`

文件	说明
<code>rootfs_hi3519dv500_64k.jffs2</code>	JFFS2 镜像 (64k erase block)
<code>rootfs_hi3519dv500_128k.jffs2</code>	JFFS2 镜像 (128k erase block)
<code>rootfs_hi3519dv500_256k.jffs2</code>	JFFS2 镜像 (256k erase block)
<code>rootfs_hi3519dv500_2k_128k_32M.ubifs</code>	UBIFS 镜像 (32M)
<code>rootfs_hi3519dv500_4k_256k_50M.ubifs</code>	UBIFS 镜像 (50M)
<code>rootfs_hi3519dv500_96M.ext4</code>	EXT4 镜像 (96M)

5.3 板端工具 (aarch64 交叉编译)

路径: `pub/bin/board_musl_arm64/`

包含: mtd-utils 工具集 (flash_erase, nandwrite, ubiformat 等)、e2fsprogs (mkfs.ext2/3/4)、ethtool 等。

5.4 PC 端工具

路径: `pub/bin/pc/`

包含: mkfs.cramfs, mkfs.jffs2, mkfs.ubifs, mkfs.ext4, ubinize, mksquashfs, nand_product。

6. 常见问题与解决方案

6.1 autoreconf: command not found

出现阶段：编译 gzip 时

原因：缺少 autoconf 包

解决：

```
apt-get install -y autoconf automake libtool
```

6.2 You must have autopoint installed

出现阶段：编译 util-linux 时

原因：缺少 gettext 包

解决：

```
apt-get install -y autopoint gettext
```

6.3 The program 'po4a' was not found

出现阶段：编译 xz 时

原因：po4a 是翻译工具链依赖

解决：

```
apt-get install -y po4a
```

6.4 linux-5.10.221.tar.gz: Cannot open: No such file or directory

出现阶段：编译 kernel 时

原因：SDK 分发包未包含内核源码压缩包

解决：

```
cd ${SDK_ROOT}/package/open_source/linux/  
wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.10.221.tar.gz
```

6.5 blkid/blkid.h: No such file or directory

出现阶段：编译 eudev 时

原因：eudev 依赖 libblkid，但交叉编译工具链 sysroot 中未提供

解决：在 `${SDK_ROOT}/package/open_source/eudev/Makefile` 的 `hi3519dv500 configure` 命令中添加 `--disable-blkid` 参数（详见 2.4 节）

6.6 ModuleNotFoundError: No module named 'Crypto'

出现阶段：打包 boot_image.bin 时（gslboot_build 步骤）

原因：Python 缺少 pycryptodome 模块

解决：

```
pip install pycryptodome
```

6.7 编译失败后重新编译提示 "Nothing to be done"

原因：Make 认为目标已完成（残留的中间文件），需清理对应组件

解决：清理失败的组件后重新编译：

```
# 示例：清理 util-linux
rm -rf ${SDK_ROOT}/package/open_source/util-linux/util-linux-2.37.2 \
    ${SDK_ROOT}/package/open_source/util-linux/out

# 或整体清理
make clean
```

7. SDK 内部参考文档

文档	路径（相对于 SDK_ROOT）	内容
eudev 编译说明	package/open_source/eudev/readme.txt	gperf 依赖安装说明
mbedtls 编译说明	package/open_source/mbedtls/readme_cn.txt	硬化方案配置、算法清单
securec 说明	package/platform/securec/README.md	安全 C 库说明
SDK 解包脚本	sdk.unpack	SDK 初始解包流程
SDK 清理脚本	sdk.cleanup	SDK 清理
BSP 顶层 Makefile	package/smp/a55_linux/source/bsp/Makefile	完整编译目标定义

8. 完整操作清单（一键准备脚本）

在 Docker 容器中首次编译前，以 root 执行以下命令完成全部准备工作：

```
#!/bin/bash
set -e

# ===== 请根据实际环境修改以下变量 =====
SDK_ROOT=<你的SDK解压路径>
# 例如：SDK_ROOT=/home/user/Hi3519DV500_SDK_V1.0.2.0
```

1. 安装系统依赖

```
apt-get update && apt-get install -y \  
    autoconf automake libtool \  
    autopoint gettext \  
    po4a \  
    gperf \  
    python3-pip
```

2. 安装 Python 依赖

```
pip install pycryptodome
```

3. 下载缺失的内核源码

```
cd ${SDK_ROOT}/package/open_source/linux/  
if [ ! -f linux-5.10.221.tar.gz ]; then  
    wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.10.221.tar.gz  
fi
```

4. 修改 eudev Makefile 禁用 blkid 依赖

```
EUDEV_MAKEFILE=${SDK_ROOT}/package/open_source/eudev/Makefile  
if ! grep -q 'disable-blkid' "$EUDEV_MAKEFILE"; then  
    sed -i 's/--disable-introspection --disable-selinux 1>\dev\|null/--disable-  
introspection --disable-selinux --disable-blkid 1>\dev\|null/' "$EUDEV_MAKEFILE"  
fi  
  
echo "=== 准备完成 ==="
```

准备完成后，以普通用户执行编译：

```
cd ${SDK_ROOT}/package/smp/a55_linux/source/bsp  
make all
```